Nous avons déjà utilisé la fonction `plot` pour tracer quelques graphiques de fonctions en 2D. Nous allons voir que MATLAB permet de faire des graphiques qui peuvent être très élaborés en 2D, ou en 3D. Dans ce chapitre, nous montrerons les fonctions et leurs paramètres les plus courants. MATLAB est d'une grande richesse pour les graphiques, aussi il faudra souvent se référer à l'aide en ligne pour avoir une description complète des commandes. .

## 1 Généralités

Les objets graphiques sont représentés dans une fenêtre différente de la fenêtre de commandes MATLAB. Cette fenêtre dispose d'une barre d'outils interactive pour la gestion des figures. Par défaut, une seule fenêtre est ouverte. Le résultat d'une nouvelle commande graphique écrasera les anciens résultats. Chaque fenêtre reçoit un numéro qui apparait en entête. Pour réactiver une fenêtre devenue inactive il suffit de taper `figure(n)` où $n$ est le numero de la figure. Pour fermer une fenêtre devenue inutile on tape `close(n)`.

## 2 Courbes en 2d

La commande principale de MATLAB pour dessiner des figures planes est l'instruction `plot(x,y)`. Cette commande permet de tracer une ligne brisée en reliant par des segments de droites les points de coordonnées `(x(i),y(i))`.

```
x=-4:0.1:4;
y=x.*sin(x).*cos(x);
plot(x,y)
```

Lorsqu'on trace plusieurs courbes sur une même fenêtre, on aimerait pouvoir les distinguer : la commande `plot` dispose d'un troisième paramètre permettant de choisir la couleur, le symbole du point, le type de tracé. Ce troisième paramètre est une chaîne de caractères de trois caractères `cst`, où $c$ est la couleur, $s$ le symbole et $t$ le style du trait.
Pour les couleurs :

| lettre | $k$ | $b$ | $g$ | $c$ | $r$ | $m$ | $y$ | $w$ | ... |
|---|---|---|---|---|---|---|---|---|---|
| couleur | noir | bleu | vert | cyan | rouge | magenta | jaune | blanc | ... |

Pour les symboles :

| caractère | . | + | × | * | $d$ | ... |
|---|---|---|---|---|---|---|
| marque | • | + | × | * | ♦ | ... |

Pour le style :

| caractère | − | : | −. | −− |
|---|---|---|---|---|
| marque | trait plein | pointillé court | pointillé long | pointillé mixte |

```
x=-4:0.1:4;
t=-4:0.5:4;
y=x.*sin(x).*cos(x);
z=exp(sin(t));
plot(x,y,'k-',t,z,'c*-')
plot(x,y,'r.-',t,z,'c*-')
```

Si différentes courbes doivent être tracées l'une après l'autre il faut maintenir la fenêtre précédente avec l'instruction `hold on` et terminer par `hold off`

```
>> x=-4:0.1:4; t=-4:0.5:4; y=x.*sin(x).*cos(x); z=exp(sin(t));
>> plot(x,y,'r.-');
>> hold on
>> plot(t,z,'c*-')
>> hold off
```

### 2.1 Les axes

La fonction `axis([x_min x_max y_min y_max])` permet de régler les axes. On en fixe les bornes min et max.
Les commandes `xlabel` et `ylabel` permettent de mettre des légendes sur les axes.

```
theta=-2:0.1:2;
y=exp(sin(theta));
plot(theta,y)
xlabel('angle \theta')
ylabel('y=exp(sin\theta)')
```

On peut aussi changer la taille et le style des caractères utilisés dans la légende. Par exemple:

```
>> xlabel('x','fontsize',14,'fontweight','b')
```

permet d'avoir l'axe des $x$ légendé avec des caractères de taille 14 en gras.
On peut éviter les commandes en lignes en utilisant la barre de menu de la fenêtre du graphique.
La fonction `axis` permet de préciser explicitement la zone graphique que l'on souhaite tracer. Cette fonction prend pour argument un vecteur donnant les valeurs extrêmes des abscisses et ordonnées.

```
x = 0:2*pi/100:2*pi;
plot(x,cos(x),x,sin(x));
axis([3 5 -1 0]);
```

### 2.2 Les commentaires

Il est possible de donner un titre à la figure, de légender les courbes et d'insérer des commentaires dans la figure.
Exemple : courbes multiples légendées. Les courbes représentent les polynômes de Tchebychev :

$$T_n(x) = \cos(n \arccos(x)),\ x \in [-1, 1],\ n = 0, 1, 2, 3$$

```
x=-1:0.01:1;
y0=ones(size(x));
y1=cos(acos(x));
y2=cos(2*acos(x));
y3=cos(3*acos(x));
plot(x,y0,':',x,y1,'-',x,y2,'--',x,y3,'-')
xlabel('x')
ylabel('y=T_n(x)')
title('Polynomes de Tchebychev')
legend('n=0','n=1','n=2','n=3')
```

### 2.3 Décomposition de la fenêtre en sous-fenêtres

Il est possible de décomposer une fenêtre en plusieurs sous-fenêtres et d'afficher une figure différente dans chaque sous-fenêtre grâce à `subplot(m,n,p)`

- $m$ est le nombre de sous-fenêtres verticales
- $n$ est le nombre de sous-fenêtres horizontales
- $p$ est la position de la sous-fenêtre à tracer

Exemple: avec trois sous-fenêtres

```
x=-4:0.1:4;
t=-4:0.1:4;
y=x.*sin(x).*cos(x);
subplot(2,2,1)
plot(x,y)
title('courbe1')
subplot(2,2,2)
z=exp(sin(t));
plot(t,z)
title('courbe2')
subplot(2,2,3)
w=exp(cos(t));
plot(t,w)
title('courbe3')
```

### 2.4 Sauvegarder une figure

La commande `print` permet de sauvegarder une figure sous différents formats.
La syntaxe est :

```
print -f<num> -d<format> <nomfich>
```

- `num` est le numero de la fenêtre graphique à sauvegarder
- `format` est le format de sauvegarde
- `nomfich` est le nom du fichier de sauvegarde

### 2.5 Courbes de niveau

Le tracé des courbes de niveau se fait par la fonction :

```
contour(x,y,y,{n})
```

- `x,y,z` représente le maillage de la surface
- `n` est le nombre de courbes de niveau à tracer

Pour tracer les courbes de niveau d'une fonction $f(x,y)$ on procède comme suit. L'exemple est fait sur la fonction :

$$f(x,y) = xye^{-x^2-y^2}, \quad (x,y) \in [-2,2]^2$$

pour faire un maillage d'une surface

```
[x,y]=meshgrid(-2:0.05:2);
z=x.*y.*exp(-x.^2 -y.^2);
contour(x,y,z,15)
```

# 3 Graphes en 3D

Il faut distinguer les courbes 3D (plus faciles à tracer) et les surfaces.
Pour les surfaces, il faut distinguer celles définies de manière analytique (par une fonction) et celles définies par un ensemble de points.

## 3.1 Courbes en 3D

On utilise la commande `plot3(x,y,z)`
où $x, y, z$ peuvent être des matrices ou des vecteurs de **même taille**.

- Dans le cas de vecteurs : la commande génère une ligne 3D qui passe par les points de coordonnées $(x_i, y_i, z_i)$
- Dans le cas de matrices de même dimension : la commande trace une courbe 3D pour chaque colonne des matrices

Exemple pour des vecteurs :

```
t=0:pi/50:10*pi;
x=(1+sin(t)).*cos(t);
y=(1+sin(t)).*sin(t);
z=t;
plot3(x,y,z)
grid on %pour la quadrillage
xlabel('x')
ylabel('y')
zlabel('z')
```

Exemple pour des matrices :

```
[x,y]=meshgrid(-2:0.1:2);
z=x.*exp(-x.^2-y.^2);
plot3(x,y,z)
```

La fonction `meshgrid` génère une grille, un maillage.

## 3.2 Surfaces analytiques

On suppose que la surface est définie par :

$$z = f(x, y)$$

- On commence d'abord par générer une grille avec la fonction `meshgrid`
  `[x,y]= meshgrid(x_min:hx:x_max, y_min:hy:y_max)`
- Ensuite, on évalue la fonction aux noeuds de cette grille pour avoir la troisième matrice $z$.
  ceci se fait à l'aide des fonctions `mesh(x,y,z)` ou `surf(x,y,z)`

```
[x,y]=meshgrid(-2:0.01:2);
z=x.*exp(-x.^2-y.^2);
mesh(x,y,z)
```

### 3.3 Surface définie par un ensemble de points

MATLAB ne sait tracer que des surfaces définies par une grille de points. Mais en pratique les surfaces à tracer sont souvent sous la forme d'un ensemble de points $x, y, z$. La visualisation de la surface nécessite un traitement préalable. Le but de ce traitement est de recréer une grille de points. D'abord on récupère les coordonnées `min` et `max` pour construire le domaine :

```
xmin = min(x); xmax=max(x);
ymin=min(y); ymax=max(y);
```

Ensuite à l'aide de `linspace` on génère une subdivition uniforme en $x$ et $y$:

```
xu=linspace(xmin,xmax,m);
yu=linspace(ymin,ymax,m);
```

où $m$ est la taille du vecteur à générer. Il suffit maintenant de générer la grille avec `meshgrid`. Et de calculer la valeur de la fonction aux noeuds de la grille. Puis pour tracer :

```
[X,Y]=meshgrid(xu,yu); % génération de la grille
Z=griddata(x,y,z,X,Y,'cubic'); % calcul de Z par interpolation
mesh(X,Y,Z) % visualisation
```

Exemple :

```
xy = -2.5 + 5*rand([200 2]);
%200 random points between -2.5 and 2.5.
x = xy(:,1);
y = xy(:,2);
v = x.*exp(-x.^2-y.^2);
%x, y, and v are vectors containing scattered  sample
%points and data.
%Define a regular grid and interpolate the scattered data
%over the grid.
[xq,yq] = meshgrid(-2:.2:2, -2:.2:2);
vq = griddata(x,y,v,xq,yq);
mesh(xq,yq,vq)
%Plot gridded data as a mesh and scattered data as dots.

plot3(x,y,v,'o')
```

Exemple :

```
x = -1:0.05:1;
[X,Y] = meshgrid(x);
Z = sin(2*pi * X.*Y);
mesh(X, Y, Z)
```

Il existe aussi la fonction `surf` qu'on laisse découvrir avec l'aide. Exemple :

```
[X,Y] = meshgrid(1:0.5:10,1:20);
Z = sin(X) + cos(Y);
surf(X,Y,Z)
```

## 4 Diagrammes pour les statistiques

MATLAB permet de tracer directement les diagrammes du type : diagramme en bâtons, camembert, histogrammes.
Pour tracer des diagrammes en bâtons, on utilise les fonctions `bar` (vertical) ou `barh` (horizontal)
Exemple :

```
x = 1900:10:2000;
y = [75 91 105 123.5 131 150 179 203 226 249 281.5];
bar(x,y)
```

Exemple :

```
X = categorical({'Small','Medium','Large','Extra Large'});
X = reordercats(X,{'Small','Medium','Large','Extra Large'});
Y = [10 21 33 52];
bar(X,Y)
```

Exemple :

```
x = [1990 2000 2010];
y = [40 50 63 52; 42 55 50 48; 30 20 44 40];
barh(x,y)
xlabel('Neige')
ylabel('Année')
legend({'Montréal','Québec','Ottawa','Toronto'})
```

Pour représenter les effectifs d'un caractère qualitatif on utilise plutôt des diagrammes appelés 'camembert'.
Exemple : Pour cela on peut faire appel à la fonction `pie`

```
X = 1:3;
labels = {'Taxes','Dépenses','Bénéfices'};
pie(X,labels)
```

Pour les séries statistiques regroupées par intervalles on utilisera la commande `histogram`
Exemple :

Gaussienne

```
x = randn(10000,1);
h = histogram(x)
```